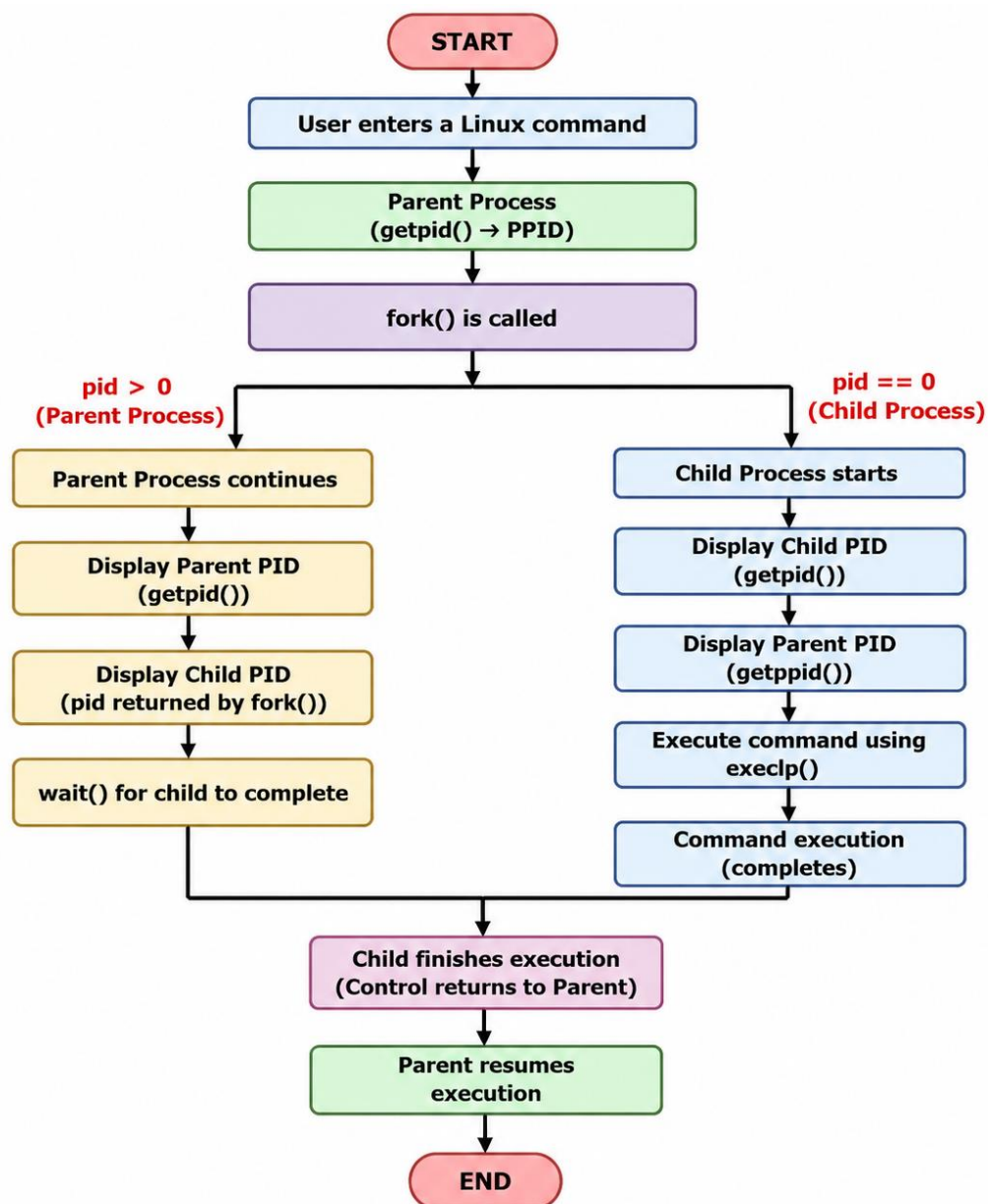


Practical Week -1

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that

1. Accept a Linux command as input.
2. Create a child process using `fork()`.
3. Execute the command in the child process using an appropriate `exec()` system call.
4. Allow the parent process to wait for the child using `wait()`.
5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using `fork()` system call.
- b) Read the user input as shell command.
- c) Use a system call `exec()` to execute the shell command.
- d) Use a `wait()` system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.



Example Output:

Enter a Linux command: ls

Parent Process

Child Process

Parent PID : 67

Child PID : 68

Child PID : 68

Parent PID : 67

Executing command: ls

CE1 FIRST.c File2.c Proces_States.c command_executor
filecopy.c hello.c
process_example session_4_a session_4_b.c

CE1.c File1.c First.c Process_States.c command_executor.c
fork_example.c.save my_shell process_example.c session_4_a.c

FIRST File1.txt Proces_States R1 filecopy hello
os-lab process_example.cyes session_4_b

Child process completed.

Sol:

GNU nano 8.7.1

command_executor.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    else if (pid == 0)
    {
        // Child process

        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("Executing command: %s\n", command);

        execlp(command, command, NULL);
```

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

```
}

else if (pid == 0)
{
    // Child process

    printf("Child Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID : %d\n", getppid());

    printf("Executing command: %s\n", command);

    execlp(command, command, NULL);

    perror("exec failed");
    exit(1);
}

else
{
    // Parent process

    printf("Parent Process\n");
    printf("Parent PID : %d\n", getpid());

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}
```

 ^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
File1.txt          ecec_demo.c  fork_exec.c  task2.c
File2.txt          exec_demo    namedirectory task3
OSSP               exec_demo.c  skill4       task3.c
OS_LAB            filename     task1
command_executor.c save filename.c  task1.c
ecec_demo          fork_exec    task2
arikel@DESKTOP-AMUSJME:~$ nano command_executor.c
arikel@DESKTOP-AMUSJME:~$ gcc command_executor.c -o command_executor
arikel@DESKTOP-AMUSJME:~$ ./command_executor
Enter a Linux command: ls
Parent Process
Parent PID : 550
Child Process
Child PID : 551
Parent PID : 550
Executing command: ls
File1.txt          ecec_demo.c  skill4
File2.txt          exec_demo    task1
OSSP               exec_demo.c  task1.c
OS_LAB            filename     task2
command_executor   filename.c   task2.c
command_executor.c fork_exec    task3
command_executor.c.save fork_exec.c  task3.c
ecec_demo          namedirectory
Child process completed.
arikel@DESKTOP-AMUSJME:~$ ./command_executor
Enter a Linux command: pwd
Parent Process
Parent PID : 617
Child Process
Child PID : 620
Parent PID : 617
Executing command: pwd
/home/arikel
Child process completed.
arikel@DESKTOP-AMUSJME:~$ |
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Sol:

Command	What it shows
uname	Linux/system information
uname -a	Detailed kernel/system information
lscpu	CPU architecture and processor information
ps	Current processes
ps aux	Detailed process information
top	Real-time process and resource usage

Command

What it shows

```

arikel@DESKTOP-AMUSJME: ~$ uname
Linux
arikel@DESKTOP-AMUSJME: ~$ uname -a
Linux DESKTOP-AMUSJME 6.18.33.2-microsoft-standard-WSL2 #
REEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Li
arikel@DESKTOP-AMUSJME: ~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits vi
Byte Order:                  Little Endian
CPU(s):                      12
On-line CPU(s) list:        0-11
Vendor ID:                   GenuineIntel
Model name:                  13th Gen Intel(R) Core(TM) i
CPU family:                  6
Model:                      186
Thread(s) per core:         2
Core(s) per socket:         6
Socket(s):                   1
Stepping:                    3
BogoMIPS:                    4991.99
Flags:                       fpu vme de pse tsc msr pae m
apic sep mtrr pge mca cmov p
6 clflush mmx fxsr sse sse2
yscall nx pdpe1gb rdtscp lm
t_tsc rep_good nopl xtopolog
eliable nonstop_tsc cpuid ts
_freq pni pclmulqdq vmx ssse
x16 pcid sse4_1 sse4_2 x2api
popcnt tsc_deadline_timer a
e avx f16c rdrand hypervisor
m abm 3dnowprefetch ssbd ibr
stibp ibrs_enhanced tpr_shad
vpid ept_ad fsgsbase tsc_adj
1 avx2 smep bmi2 erms invpci
d adx smap clflushopt clwb s
saveopt xsavec xgetbv1 xsave

```

```
arikela@DESKTOP-AMUSJME: ~$ ps
  PID TTY          TIME CMD
  319 pts/0    00:00:00 bash
  733 pts/0    00:00:00 ps
arikela@DESKTOP-AMUSJME: ~$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TI
ME COMMAND
root           1  0.3  0.3   24328 15372 ?        Ss   05:20   0:
03 /sbin/init
root           2  0.0  0.0    3180   2208 hvc0     Sl+  05:20   0:
00 /init
root           7  0.0  0.0    3180   2048 hvc0     Sl+  05:20   0:
00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-f
root          53  0.0  0.4  50396 16804 ?        S<s  05:20   0:
00 /usr/lib/systemd/systemd-journald
systemd+       70  0.0  0.3   22416 14468 ?        Ss   05:20   0:
00 /usr/lib/systemd/systemd-resolved
root           92  0.1  0.3   35140 12256 ?        Ss   05:20   0:
01 /usr/lib/systemd/systemd-udev
root          153  0.0  0.0    2888   1948 ?        Ss   05:20   0:
00 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
root          154  0.0  0.0    4472   3100 ?        Ss   05:20   0:
00 /usr/sbin/cron -f -P
message+       155  0.0  0.1    8820   5316 ?        Ss   05:20   0:
00 @dbus-daemon --system --address=systemd: --nofork --nopidfile
root          159  0.0  0.7  44096 29712 ?        Ss   05:20   0:
00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-t
root          168  0.0  0.2   18436   9388 ?        Ss   05:20   0:
00 /usr/lib/systemd/systemd-logind
syslog        175  0.0  0.1  220548   5568 ?        Ssl  05:20   0:
00 /usr/sbin/rsyslogd -n -iNONE
root          216  0.0  0.0    5492   2660 tty1     Ss+  05:20   0:
00 /usr/sbin/agetty --noreset --noclear --issue-file=/etc/issue:
root          225  0.0  0.8 123460 32772 ?        Ssl  05:20   0:
00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-up
_chrony        226  0.0  0.2   23588  11084 ?        S    05:20   0:
00 /usr/sbin/chronyd -n -F 1 -x
```




arikela@DESKTOP-AMUSJME: X



arikela@DESKTOP-AMUSJME:~\$ top

top - 05:43:42 up 2:51, 1 user, load average: 0.09, 0.04, 0.
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 z
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi,
MiB Mem : 3758.6 total, 3048.4 free, 473.1 used, 312.
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 3285.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
1	root	20	0	24328	15372	11568	S	0.0	0.4
2	root	20	0	3180	2208	2072	S	0.0	0.1
7	root	20	0	3180	2048	1940	S	0.0	0.1
53	root	19	-1	50396	16804	15500	S	0.0	0.4
70	systemd+	20	0	22416	14468	11972	S	0.0	0.4
92	root	20	0	35140	12256	9184	S	0.0	0.3
153	root	20	0	2888	1948	1820	S	0.0	0.1
154	root	20	0	4472	3100	2852	S	0.0	0.1
155	message+	20	0	8820	5316	4636	S	0.0	0.1
159	root	20	0	44096	29712	14576	S	0.0	0.8
168	root	20	0	18436	9388	8144	S	0.0	0.2
175	syslog	20	0	220548	5568	4648	S	0.0	0.1
216	root	20	0	5492	2660	2476	S	0.0	0.1
225	root	20	0	123460	32772	18164	S	0.0	0.9
226	_chrony	20	0	23588	11084	7128	S	0.0	0.3
237	_chrony	20	0	12096	2452	1808	S	0.0	0.1
316	root	20	0	3184	1112	980	S	0.0	0.0
317	root	20	0	3200	1248	1108	S	0.0	0.0
319	arikela	20	0	6268	5532	3864	S	0.0	0.1
322	root	20	0	8648	5544	4700	S	0.0	0.1
385	arikela	20	0	22340	13604	10976	S	0.0	0.4
387	arikela	20	0	22920	4092	2088	S	0.0	0.1
412	arikela	20	0	6136	5424	3864	S	0.0	0.1
755	arikela	20	0	8164	6064	3948	R	0.0	0.2